ggml_quantize_mat_q8_0_4x4
多平台向量实现对比与RVV扩展指令建议

RVFuse Project

2026-04-15

目 录

ggml_quantize_mat_q8_0_4x4 多平台向量实现对比与RVV扩展指令建议

- 概述
- 指令方案汇总
- 基准 RVV 实现分析
- 各平台对比分析
- RVV 扩展指令建议详细说明
- 附录
- 结论
- 审查日志

ggml_quantize_mat_q8_0_4x4 多平台向量实现对比与RVV扩展指令建议

概述

分析目标: `ggml_quantize_mat_q8_0_4x4` — 将 4 行 × 32 列 float32 矩阵量化为 int8，按 4 字节粒度 4 路交织存储（用于 GEMV/GEMM 中的权重量化预处理） 基准实现: RVV VLEN=512, SEW=32, LMUL=8 (f32 load) / LMUL=2 (i8/i32 store) 分析平台: x86 AVX/AVX-512, ARM NEON, LoongArch LASX, Power VSX (POWER10), S390X Z-Vector, WASM SIMD **BBV**数据: 未提供，收益为理论估算（BB 作用域内指令减少比例）

指令方案汇总

优先级	扩展指令	来源平台	BB内收益	实现难度	RVV现状
P0	vfncvt_x_f_w_i8 (f32i8 一步窄化转换)	x86 AVX-512 `vpmovdb`	BB内减少约9.8%（量化循环BB）	中	需 f32i16i8 两步，i16 中间结果浪费；AVX-512 f32i8 也是两步
P1	vfredmax.vs + vfmv.f.s 融合	ARM NEON `vmaxvq_f32`	BB内减少约7%（amax归约BB）	低	当前需两条指令，ARM 一条完成归约+标量提取

注: 无 BBV profiling 数据，上表仅反映单个 BB 范围内的指令减少比例，无法推算整体收益。建议通过 `./tools/profile_to_dfg.sh` 获取 BBV 数据后重新估算。

总体结论: RVV 在此 kernel 上对 ARM NEON 有 90%+ 指令数优势，对假想 AVX-512 优化版有约 25% 优势。RVV 的 `vsseg4e32.v` 段存储指令是该 kernel 的决定性优势，无需为此扩展 RVV ISA。

基准 RVV 实现分析

数据流

```
输入: float32 x[4][k], k 为 QK8_0=32 的倍数
输出: block_q8_0x4 { ggml_half d[4]; int8_t qs[128]; }

每块 (4行 × 32元素):
  对每行 (重复4次):
    vle32.v_f32m8 → 加载 32 个 float32
    vfabs.v_f32m8 → 取绝对值
    vfredmax.vs_f32m8 → 归约求最大绝对值 → amax (标量)
    标量: d = amax/127, 存 fp16, id = 1/d
    vfmul.vf_f32m8 → 乘以 id
    vfncvt.x.f.w_i16m4 → f32→i16 窄化转换 (round-to-nearest-even)
    vnsrl.wi → i16→i8 窄化 (右移0位截断)

  4 行量化完成后:
    vreinterpret_i8m2_i32m2 × 4 → 重解释为 int32 (零开销)
    vcreate_v_i32m2x4 → 创建 4 段元组
    vsseg4e32.v vl=8 → 段存储: 4段 × 8元素 × 4B = 128B
```

指令计数 (每块)

阶段	向量指令	标量指令	小计
加载 (4行)	4 × vle32	—	4
绝对值 (4行)	4 × vfabs	—	4
归约 (4行)	4 × vfredmax	4 × (fmv.f.s + div + fp16_store + id_calc)	4 + 16
缩放 (4行)	4 × vfmul.vf	—	4
f32i16 (4行)	4 × vfncvt	—	4
i16i8 (4行)	4 × vnsrl	—	4
段存储	1 × vsseg4e32	—	1
总计	**25**	**16**	**41**

寄存器使用

- `vfloat32m8_t`: 加载/计算时占用 8 个向量寄存器（逐行复用）
- `vint8m2_t` × 4: 量化结果占用 8 个向量寄存器（行间需同时保持）
- `vint32m2x4_t`: 段存储元组，复用上述 8 个寄存器
- 峰值寄存器占用: ~16/32（行内处理时 m8=8 + 结果 m2×4=8）

各平台对比分析

1. x86 AVX/AVX-512

核心特点:

- AVX2: 16 × 256-bit YMM, AVX-512: 32 × 512-bit ZMM
- 丰富的 narrowing/packing 指令 (`vpmovdb`, `vpmovdw`)
- 无段存储，interleave 依赖 shuffle 指令组合
- 当前状态: 无优化实现，直接使用标量 fallback

高价值指令:

指令	功能	**RVV** 等价	差距
`vpmovdb` (AVX-512)	int32int8 一步打包，16 元素/条	`vfncvt` + `vnsrl` 两步	RVV 每次窄化 SEW 只能减半
`vreduceps` (AVX-512)	单指令向量标量 reduce	`vfredmax` + `vfmv.f.s`	语义等价，RVV 多一步提取
`vshufps` + `vshufi32x4`	跨 lane shuffle 实现 interleave	`vsseg4e32.v`	**RVV** 优势：1 条 vs 6-8 条

假想 **AVX-512** 优化版 **vs RVV** 对比:

维度	假想 **AVX-512**	**RVV (VLEN=512)**	优势方
每行 load	2× `_mm512_loadu_ps`	1× `vle32.v_f32m8`	RVV: LMUL 跨寄存器
f32i8 narrowing	`_mm512_cvtps_epi32` + `_mm512_cvtsepi32_epi8` (2步)	`vfncvt` + `vnsrl` (2步)	持平
Interleave	6-8 条 shuffle + store	1 条 `vsseg4e32.v`	**RVV:** 省 **5-7** 条**/**块
每块总指令	~60 条	~41 条	**RVV** 少 **32%**

建议扩展:

- **[P0]** 一步 **f32i8** 窄化转换: 允许 narrowing convert 跨越 2 个 SEW 级别（328）。AVX-512 的 `vpmovdb` 可一步完成 i32i8 整数窄化（但 f32i8 仍需 `vcvtps2dq` + `vpmovdb` 两步）；RVV 的差距在于窄化仅限 SEW/2，导致 f32i16 步骤浪费（中间 i16 从未被需要）。每行省 1 条指令，4 行省 4 条，BB 内减少约 9%。

2. ARM NEON

核心特点:

- 32 × 128-bit Q 寄存器，4 个 float32/寄存器
- 无段存储，interleave 需逐 lane 提取 + 逐字节存储
- 水平归约需树形 `vmaxq` 配对（3 级）

高价值指令:

指令	功能	RVV 等价	差距
`vcvtnq_s32_f32`	f32i32 舍入转换（原生宽度）	`vfcvt.x.f.v`	无差距，但 RVV 代码直接用窄化
`vgetq_lane_s32`	提取单个 lane 到标量	`vmv.x.s` + slide	**ARM** 劣势：需 16 次/循环迭代
`vmaxvq_f32`	从 Q 寄存器提取水平最大值	`vfredmax` + `vfmv.f.s`	ARM 需先 3 级 `vmaxq` 再 `vmaxvq`

ARM NEON vs RVV 指令计数对比 **(**每块**, 4**行**×32**元素**):**

阶段	ARM NEON	RVV	比值
加载	32 条 (`vld1q` × 8/行)	4 条 (`vle32` × 1/行)	8.0×
绝对值	32 条	4 条	8.0×
水平最大值归约	32 条 (7×`vmaxq` + 1×`vmaxvq`)/行	8 条 (`vfredmax` + `vfmv.f.s`)/行	4.0×
缩放	32 条	4 条	8.0×
Convert + Lane 提取 **+** 存储	**288** 条 (32×`vcvtnq` f32i32 + 128×lane 提取 i32 + 128×byte store 隐式截断 i32i8)	**9** 条 (4×`vfncvt` + 4×`vnsrl` + 1×`vsseg4e32`)	**32.0×**
总计	**~416**	**~41**	**10.1×**

结论: ARM NEON 在此 kernel 上相对 RVV 有 10 倍指令数差距，核心瓶颈是缺乏段存储和固定 128 位宽度。RVV 无需为此 kernel 添加任何 ARM 特有能力。

3. LoongArch LASX

核心特点: 32 × 256 位向量寄存器，LASX (256-bit) / LSX (128-bit) 扩展 优化实现: 无，仅有 `quantize_row_q8_0` 单行量化的 LSX 实现 高价值指令: `vpickev_h/b`（交错打包）、`vsat_w/h`（饱和操作） 建议: 无针对此 kernel 的新扩展。LASX 的打包指令可优化窄化但 RVV 已有等价方案。

4. Power VSX (POWER10)

核心特点: 32 × 128 位 VSX 寄存器，POWER10 MMA 矩阵加速 优化实现: 无，仅有 `quantize_row_q8_0` 的 Power9 向量实现 高价值指令: `vec_pack`（i32i8 两级打包），`xvcvspbf16`（FP16 转换） 建议: Power 的 `vec_pack` 链可一次完成 i32i8，但 RVV 的两步窄化在功能上等价。

5. S390X Z-Vector

核心特点: 32 × 128 位向量寄存器，VXE/VXE2 扩展 优化实现: 无 高价值指令: `vfisb`（F32I32 直接转换） 建议: 无针对此 kernel 的 RVV 扩展建议。

6. WASM SIMD

核心特点: 128 位 `v128_t`，`wasm_simd128` 内建函数 优化实现: 无 高价值指令: `i32x4_trunc_sat_f32x4`（F32I32 饱和截断） 建议: 无针对此 kernel 的 RVV 扩展建议。

RVV 扩展指令建议详细说明

[P0] vfncvt_x_f_w_i8 — f32i8 一步窄化转换

指令定义:

- 语义: 将 float32 向量直接转换为 int8 向量，带舍入模式选择
- 编码约束: 使用现有 `vfncvt` 编码空间，目标 SEW=8 而非 SEW=SEW/2
- 行为: `dst[i] = saturate(round(src[i]))`，SEW 从 32 直接到 8

应用场景: 所有需要 float低精度整数量化的 kernel，包括 Q8_0/Q4_0/Q4_K 量化、MXFP4 转换等。

性能对比 (处理 32 个 float32int8):

```
当前 RVV (两步):
  vfncvt.x.f.w v_i16, v_f32, rm, vl // f32 → i16 (1 条)
  vnsrl.wi v_i8, v_i16, 0, vl // i16 → i8 (1 条)
  总计: 2 条

扩展后 (一步):
  vfncvt_x_f_w_i8 v_i8, v_f32, rm, vl // f32 → i8 (1 条)
  总计: 1 条

每行节省: 1 条
每块 (4行) 节省: 4 条
BB 内减少: 4 / (25 + 16) ≈ 9.8%
```

实现难度: 中。需要硬件支持跨 2 个 SEW 级别的窄化数据通路。

与 **AVX-512** 的精确对比: AVX-512 完整 f32i8 路径同样是两步（`vcvtps2dq` 做 f32i32 + `vpmovdb` 做 i32i8），并非一步完成。`vpmovdb` 的优势仅限于整数域内一步跨越 i32i8。RVV 的真正差距是窄化限定为 SEW/2，导致必须经过 f32i16 这一步，而中间 i16 值从未被使用——这一步是"浪费"的。

[P1] vfredmax + vfmv 融合 — 单指令向量最大值到标量

指令定义:

- 语义: 将 `vfredmax.vs` + `vfmv.f.s` 融合为一条指令，直接返回标量最大值

- 编码: 可使用现有 reduction 指令的变体，增加标量目标寄存器

应用场景: 所有需要 amax 计算的量化 kernel（Q8_0, Q4_0, Q8_K 等）。

性能对比:

```
当前 RVV (两步):
  vfredmax.vs v_max, v_abs, v_zero, vl // 归约到 m1 (1 条)
  vfmv.f.s amax, v_max // 提取标量 (1 条)
  总计: 2 条

扩展后 (一步):
  vfredmax_to_scalar amax, v_abs, v_zero, vl // 直接归约到标量 (1 条)
  总计: 1 条

每行节省: 1 条
每块 (4行) 节省: 4 条
BB 内减少: 4 / 41 ≈ 9.8%
```

实现难度: 低。微架构上 reduction 本就需要将结果写回标量寄存器，仅是编码层面的融合。

附录

A. 交织存储指令对比表

平台	指令	粒度	指令数/块	备注
RVV	`vsseg4e32.v`	4×4 字节	1	单指令完成 128 字节 4 路交织
x86 AVX-512	`vshufps` + `vshufi32x4` + store	4 字节	6-8	多条 shuffle 组合
ARM NEON	`vgetq_lane` + scalar store	1 字节	256	逐 lane 提取 + 逐字节写入
Power VSX	`vec_pack` + 逐元素	1-2 字节	~32	打包后仍需逐元素
LoongArch	`vpickev` + 逐元素	1-2 字节	~32	类似 Power
S390X	无专用指令	—	~64	标量
WASM SIMD	`i32x4.extract_lane`	1 字节	~128	类似 ARM

B. Narrowing 转换指令对比表

平台	**f32i8** 路径	步骤数	指令
RVV	f32i16i8	2	`vfncvt` + `vnsrl`
x86 AVX-512	f32i32i8	2	`vcvtps2dq` + `vpmovdb`
ARM NEON	f32i32(隐式截断到 i8)	1+隐式	`vcvtnq_s32_f32` + lane 提取
Power VSX	f32i32i16i8	3	`xvrdpim` + `vpack` × 2
LoongArch LASX	f32i32i16i8	3	`vftintrne.w.s` + `vpickev` × 2

C. 加载/存储效率对比表 (每块 128 字节输出)

平台	寄存器宽度	每行加载次数	每块存储指令数
RVV (VLEN=512)	512-bit	1 (LMUL=8)	1 (段存储)

平台	寄存器宽度	每行加载次数	每块存储指令数
x86 AVX-512	512-bit	2	6-8 (shuffle+store)
ARM NEON	128-bit	8	256 (逐 lane)
LoongArch LASX	256-bit	4	~64
Power VSX	128-bit	8	~64

结论

主要发现

1. **RVV** 在此 **kernel** 上具有全面优势，尤其是 `vsseg4e32.v` 段存储指令，单条指令替代 ARM NEON 的 256 条 lane 提取 + 标量存储操作。
2. 唯一的实质差距是 RVV 窄化每次仅限 SEW/2（f32i16i8），其中 f32i16 步骤的中间 i16 结果从未被需要；而 AVX-512 在整数域可一步完成 i32i8（`vpmovdb`），但完整 f32i8 路径同样是两步（`vcvtps2dq` + `vpmovdb`）。RVV 的真正差距在于无法跳过不需要的中间 SEW 级别。影响有限（每块多 4 条指令，约 9.8%）。
3. **ARM NEON** 是差距最大的平台（10.1× 指令数差距），其固定 128 位宽度和缺乏段存储是根本瓶颈。

优先级建议

优先级	建议	理由
1	保持当前 RVV 实现，不修改	当前实现已是最优，vsseg4e32 是决定性优势
2	探索 P0 (一步 f32i8 窄化)	如硬件支持可减少约 9% 指令，但收益有限
3	P1 (reduction+标量提取融合)	收益仅 ~10%，实现简单但动机不足

审查日志

轮次	发现问题数	已修复	剩余
R1	2 (minor)	2	0
R2	2 (minor)	2	0

R1 问题: ARM NEON 总数 ~428~416（修正计数）， P0 描述澄清 AVX-512 也是两步 **R2** 问题: P0 百分比 "约9%""约9.8%"（统一精度）， ARM 表格补充隐式截断说明

最终审查结论：R2 所有问题已修复，报告准确。